

ReadingsController.mjs

This file answers:

What is the client asking us to do?

Your controller imports functions from the service:

```
1 import {
2   AppError,
3   createReading,
4   deleteReading,
5   getBranchReadings,
6   getReadingById,
7   patchReading
8 } from "../services/readingsService.mjs";
```

This is the first major connection:

```
1 CONTROLLER
2   |
3   | imports
4   ▼
5 SERVICE
```

The controller doesn't actually know how DynamoDB saves something.

It just knows:

```
1 POST → createReading()
2
3 GET all → getBranchReadings()
4
5 GET one → getReadingById()
6
7 PATCH → patchReading()
8
9 DELETE → deleteReading()
```

Response helper

Your controller has something similar to:

```
1 function response(
2   statusCode,
3   body = null
4 ) {
5
6   if (statusCode === 204) {
7
8     return {
9       statusCode: 204,
10      body: ""
11    };
12  }
13 }
```

```

14
15     return {
16         statusCode,
17
18         headers: {
19             "Content-Type":
20                 "application/json"
21         },
22
23         body:
24             JSON.stringify(body)
25     };
26 }

```

This creates the response API Gateway expects.

For example:

```

1 response(
2     200,
3     reading
4 );

```

becomes roughly:

```

1 {
2     "statusCode": 200,
3     "headers": {
4         "Content-Type": "application/json"
5     },
6     "body": "{ ... }"
7 }

```

API Gateway then sends that back to Postman.

So:

```

1 Service result
2   ↓
3 Controller
4   ↓
5 response()
6   ↓
7 API Gateway
8   ↓
9 Postman

```

Why is 204 different?

DELETE returns:

```

1 204 No Content

```

That means there should be no JSON response body.

That's why your helper has:

```

1 if (statusCode === 204) {
2     return {
3         statusCode: 204,

```

```
4     body: ""
5   };
6 }
```

parseBody()

You also have something similar to:

```
1 function parseBody(event) {
2
3   try {
4
5     return JSON.parse(
6       event.body ?? "{}"
7     );
8
9   } catch {
10
11     throw new AppError(
12       400,
13       "Request body must be valid JSON"
14     );
15   }
16 }
```

Remember that API Gateway gives Lambda the HTTP body as **text**.

Postman sends:

```
1 {
2   "maxTempC": 12
3 }
```

but Lambda may receive:

```
1 event.body =
2   '{"maxTempC":12}'
```

That's a string.

So:

```
1 JSON.parse(event.body)
```

turns it back into:

```
1 {
2   maxTempC: 12
3 }
```

Now JavaScript can use:

```
1 body.maxTempC
```

handleRequest()

This is the main function in the controller.

It receives:

```
1 event
2 context
```

from `index.mjs`.

It first logs useful information:

```
1 console.log(
2   "REST API request received",
3   {
4     requestId:
5       context.awsRequestId,
6
7     httpMethod:
8       event.httpMethod,
9
10    resource:
11      event.resource
12  }
13 );
```

That goes to:

```
1 CloudWatch Logs
```

Building the route

The controller does something like:

```
1 const httpMethod =
2   event.httpMethod;
3
4 const resource =
5   event.resource;
6
7 const route =
8   `${httpMethod} ${resource}`;
```

Suppose Postman sends:

```
1 DELETE
2 /branches/BRANCH-001/readings/ABC
```

API Gateway knows the resource template is:

```
1 /branches/{branchId}/readings/{readingId}
```

So:

```
1 route
```

becomes:

```
1 DELETE /branches/{branchId}/readings/{readingId}
```

Reading path parameters

Then:

```
1 const branchId =  
2   event.pathParameters?.branchId;  
3  
4 const readingId =  
5   event.pathParameters?.readingId;
```

For:

```
1 /branches/BRANCH-001/readings/1234
```

you get:

```
1 branchId  
2 =  
3 BRANCH-001  
4  
5 readingId  
6 =  
7 1234
```

The switch statement

This is one place where `switch` is very useful:

```
1 switch (route) {
```

because there are several known possible routes.

GET all

```
1 case "GET /branches/{branchId}/readings":
```

The controller reads:

```
1 from  
2 to
```

from query parameters and calls:

```
1 getBranchReadings({  
2   branchId,  
3   from,  
4   to  
5 });
```

Notice the important connection:

```
1 Controller  
2  
3 getBranchReadings()
```

```
4      |
5      ▼
6  Service
7
8  export async function
9  getBranchReadings()
```

GET one

```
1 | case "GET /branches/{branchId}/readings/{readingId}":
```

calls:

```
1 | getReadingById(
2 |   branchId,
3 |   readingId
4 | );
```

POST

```
1 | case "POST /branches/{branchId}/readings":
```

first parses:

```
1 | const body =
2 |   parseBody(event);
```

then:

```
1 | const reading =
2 |   await createReading(
3 |     branchId,
4 |     body
5 |   );
```

and returns:

```
1 | response(
2 |   201,
3 |   reading
4 | );
```

PATCH

PATCH does:

```
1 | const body =
2 |   parseBody(event);
```

then:

```
1 | patchReading(
2 |   branchId,
3 |   readingId,
4 |   body
```

```
5 );
```

and returns:

```
1 200
```

DELETE

DELETE calls:

```
1 deleteReading(  
2   branchId,  
3   readingId  
4 );
```

and returns:

```
1 204
```

Controller error handling

The whole switch is inside a:

```
1 try {
```

Then errors are caught:

```
1 catch (error) {
```

Your custom:

```
1 AppError
```

allows the service to say things like:

```
1 400  
2 404  
3 409
```

For example:

```
1 throw new AppError(  
2   404,  
3   "Reading not found"  
4 );
```

The controller sees that and sends:

```
1 {  
2   "error": "Reading not found"  
3 }
```

with:

```
1 404
```

Unexpected errors become:

```
1 | 500 Internal Server Error
```

So the service decides **what went wrong**, and the controller decides **how that becomes an HTTP response**.